Sistem de urmărire automată a unui obiect cu ajutorul unei drone

Documentație de proiect - Milestone 1 · IoT & IOC

1. Introducere

Dronele sunt folosite tot mai des pentru filmări, supraveghere, inspecții sau operațiuni de căutare și salvare. În multe dintre aceste situații, drona trebuie să urmărească un obiect aflat în mișcare, cum ar fi o persoană sau un vehicul. Pilotarea manuală devine atunci dificilă și obositoare, așa că mi s-a părut interesantă ideea unui sistem care să facă singur această urmărire.

Tema proiectului meu este „Sistem de urmărire automată a unui obiect cu ajutorul unei drone". Sistemul ar analiza imaginile primite de la camera dronei, ar recunoaște în ele obiectul de interes și ar mișca drona astfel încât obiectul să rămână mereu în cadru, fără ca un om să o piloteze. În același timp, ar oferi informații despre deplasarea obiectului (poziție, viteză, direcție) și le-ar transmite prin rețea către o pagină de monitorizare.

Proiectul se încadrează în tematica materiei IoT & IOC pentru că îmbină trei elemente: un dispozitiv cu senzor (drona cu cameră), prelucrarea datelor pe un calculator și comunicația între componente printr-un protocol specific IoT. Nu dispun de o dronă fizică, așa că intenționez să lucrez cu o dronă simulată. Vreau totuși să construiesc sistemul astfel încât să poată fi conectat ulterior și la o dronă reală.

Proiectul va fi dezvoltat în 5 etape, fiecare încheiată cu o prezentare a aplicației. La final, codul sursă și documentația vor fi disponibile public pe GitHub.

2. Ce îmi propun să realizez

Ca obiectiv principal, vreau ca drona să urmărească singură un obiect ales, iar omul doar să supravegheze. Concret, mi-aș dori ca sistemul să poată:

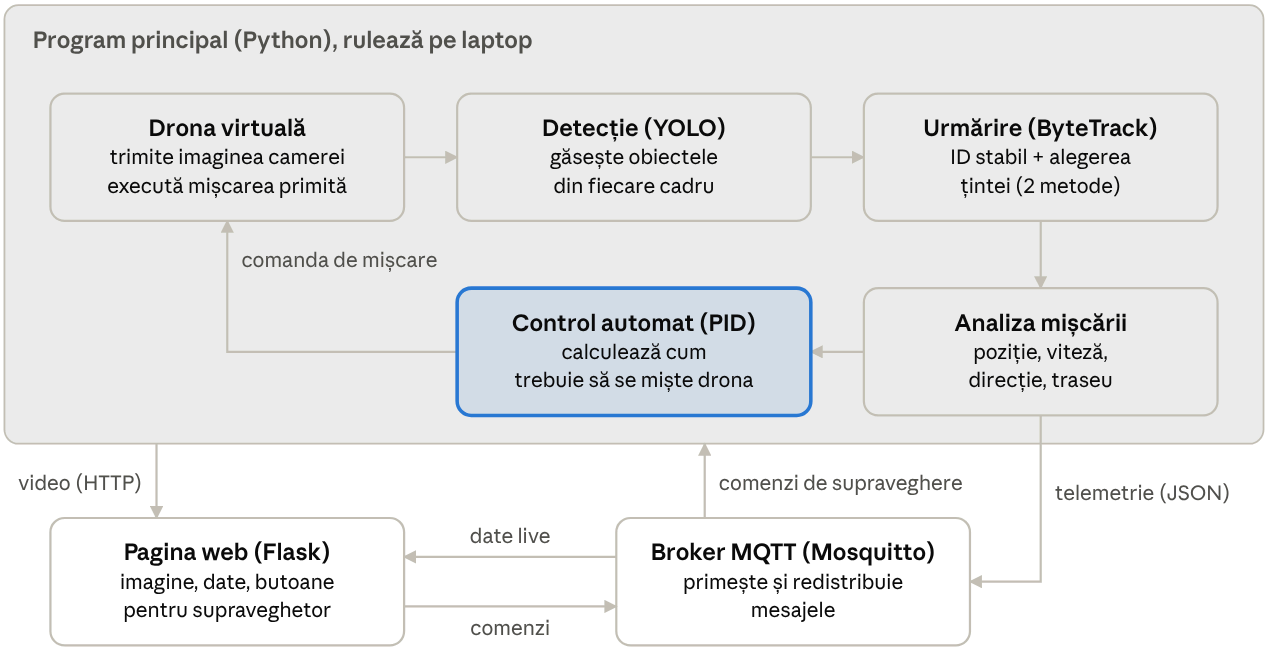
- recunoaște obiectele din imaginea camerei (persoane, mașini, biciclete etc.);
- alege ținta în două moduri: **automat**, după criterii precum tipul obiectului, culoarea și mărimea (de exemplu „persoana îmbrăcată în roșu"), sau **la indicația supraveghetorului**, printr-un click pe obiectul dorit;
- urmări ținta de la un cadru la altul, chiar dacă în imagine apar și alte obiecte asemănătoare, și o regăsi dacă dispare pentru scurt timp;

- calcula informații despre deplasarea țintei: poziție, viteză, direcție și traseul parcurs;
- mișca automat drona astfel încât ținta să rămână în centrul imaginii;
- transmite datele prin rețea către o pagină web, de unde supraveghetorul poate vedea ce se întâmplă și poate da comenzi simple: pornire, oprire, schimbarea țintei și oprire de urgență.

3. Cum aș vrea să funcționeze sistemul

Mă gândesc la sistem ca la o buclă: drona trimite o imagine, programul o analizează și decide cum trebuie să se miște drona ca ținta să rămână în centru, iar drona execută mișcarea. Ciclul s-ar repeta de mai multe ori pe secundă. În paralel, datele ar pleca prin rețea spre o pagină de monitorizare. Diagrama arată cum îmi imaginez componentele în acest moment.

Drona se mișcă singură în buclă închisă; MQTT duce datele la supraveghetor



arhitectura sistemului · 7 componente

Fiecare componentă ar urma să fie un modul separat, ca să le pot dezvolta și testa pe rând. Supraveghetorul nu pilotează drona, ci doar urmărește datele și dă comenzi simple. Este posibil ca structura să se schimbe pe parcurs, pe măsură ce implementez și testez.

4. Tehnologii pe care intenționez să le folosesc

Limbajul de programare va fi **Python**. L-am ales pentru că majoritatea bibliotecilor de prelucrare a imaginilor și de IoT îl suportă bine și pentru că există multe exemple din care pot învăța.

Tehnologie	La ce aș folosi-o
OpenCV	Citirea imaginilor, desenarea peste ele și prelucrarea culorilor
YOLO (biblioteca Ultralytics)	Recunoașterea obiectelor din imagine, cu un model gata antrenat
ByteTrack	Urmărirea aceluiași obiect de la un cadru la altul
Regulator PID	Calcularea mișcării dronei în funcție de poziția țintei
MQTT + Mosquitto	Comunicația între componente, printr-un protocol des folosit în IoT
Flask	Pagina web de monitorizare
Git + GitHub	Păstrarea și publicarea codului și a documentației

Detaliile (ce variantă de model folosesc, la ce rezoluție lucrez, cum reglez controlul) le voi stabili după primele teste și le voi descrie în documentația finală.

5. De ce o dronă simulată

Nu am o dronă fizică, iar una reală ar presupune costuri, spațiu de zbor și riscuri. De aceea vreau să creez o dronă virtuală simplă: camera dronei ar fi o „fereastră” care se deplasează peste o imagine sau un video mai mare. Când drona „se rotește”, „urcă” sau „înaintează”, fereastra se mută sau face zoom, la fel cum s-ar schimba imaginea unei camere reale.

Astfel pot testa tot sistemul pe calculator, iar testele pot fi repetate în aceleași condiții. Vreau ca restul programului să comunice cu drona printr-o interfață comună, ca o dronă reală să poată fi conectată mai târziu fără modificări mari.

6. Planul pe etape (M1–M5)

Am împărțit proiectul în 5 etape. Fiecare etapă se construiește pe cea anterioară și se încheie cu ceva care poate fi demonstrat live.

Fiecare etapă adaugă o funcție nouă și se încheie cu un demo live

M1 · Documentația

Ce implementăm

Documentul de proiect (acest document)
Repo GitHub cu structura de foldere
Alegerea bibliotecilor și uneltelor

Ce prezentăm

Scopul proiectului și obiectivele
Arhitectura propusă și tehnologiile
Planul de lucru pe etape

M2 · Detecție, urmărire și alegerea țintei

Ce implementăm

YOLO + ByteTrack pe video și webcam
Alegere automată: tip, culoare, mărime
Alegere prin click + regăsirea țintei

Ce prezentăm

Sistemul alege singur „persoana în roșu”
Trecerea pe altă țintă printr-un click
Urmărirea printre obiecte asemănătoare

M3 · Analiza mișcării și drona virtuală

Ce implementăm

Poziție, viteză, direcție, traseu
Drona virtuală (fereastra pe scenă)
Scenă de test cu traseu cunoscut

Ce prezentăm

Datele de deplasare afișate live
Drona virtuală pe un traseu prestabilit

M4 · Urmărire automată și componenta IoT

Ce implementăm

Controlul PID în buclă închisă
Mesaje MQTT prin brokerul Mosquitto
Pagina web de monitorizare (Flask)

Ce prezentăm

Drona găsește și urmărește ținta singură
Regăsirea țintei după o scurtă dispariție
Date live în browser, oprire de urgență

M5 · Prezentarea finală

Ce implementăm

Reglaj PID, stabilizare, măsurători
Documentația finală pe GitHub Pages
README cu instrucțiuni de instalare

Ce prezentăm

Demo complet pe un scenariu pregătit
Rezultate: % timp în cadru, eroare, cadre/s
Linkurile publice: cod + documentație

planul pe etape · M1–M5

7. Ce rămâne de stabilit pe parcurs

Unele decizii nu le pot lua înainte de a scrie și testa codul:

- ce variantă a modelului YOLO și ce rezoluție a imaginii pot folosi pe laptopul meu, în funcție de viteza obținută;
- cât de bine funcționează alegerea țintei după culoare în condiții diferite de lumină;
- cum reglez controlul dronei astfel încât mișcarea să fie lină, fără oscilații.

Rezultatele acestor teste le voi prezenta la etapele următoare și în documentația finală.